

ADL Lite: An Event-First Operational Ontology for Concept Lifecycle Governance

Anonymous Authors
Affiliation withheld for peer review

Abstract

This paper presents ADL Lite, an event-first operational ontology for concept lifecycle governance. Drawing on Wittgenstein’s thesis that “the world is the totality of facts, not of things,” ADL Lite treats events—not objects—as the fundamental ontological primitive. Concepts are modeled as append-only, cryptographically linked EventChains, with all lifecycle state (status, confidence, validators) derived exclusively from event history through deterministic functions. This design eliminates stored mutable state and the consistency bugs it engenders.

The paper’s primary contribution is an ontological analysis that situates ADL Lite within the landscape of foundational ontologies. We demonstrate that ADL Lite’s Event corresponds to the BFO category of *occurrent* and the DOLCE category of *perdurant*; that its Concept is best understood as a BFO *generically dependent continuant* or a DOLCE *non-physical object*; and that its Relation blocks instantiate UFO *relators*. Identity conditions for events, concepts, and chains are formalized, and three forms of ontological dependence (historical, generic, mutual) are analyzed. Alignment choices and intentional deviations from BFO, DOLCE, and UFO are discussed and justified.

A complete formal semantics is provided: an event alphabet $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$, a well-formedness predicate $\text{WF}(C)$, status derivation $\delta(C)$, confidence aggregation $\gamma(C)$, and fork/confluence semantics. Six properties are proved: determinism (Theorem 1), confluence under fork (Theorem 2), transition monotonicity (Theorem 3), confidence boundedness (Theorem 4), confidence monotonicity under independent validation (Theorem 5), and status–confidence consistency (Theorem 6). The expressive power of the derivation functions is analyzed in relation to Event Calculus and Description Logic.

Empirical validation on 201 EventChains derived from the IBM Anti-Money Laundering dataset (9,300 events) confirms zero integrity failures and linear scalability. The ontology is implemented as a pip-installable Python toolkit operating over Markdown files in standard Git repositories, with no external triple store or OWL reasoner required.

Keywords: ontology engineering; event sourcing; conceptual modeling; applied ontology; multi-agent systems; knowledge graphs; provenance; BFO; DOLCE; UFO

1 Introduction

1.1 Background: From Object-First to Event-First Ontology Engineering

Ontology engineering has traditionally adopted an *object-first* stance. In this paradigm, the ontologist begins by identifying the kinds of entities that exist in a domain (Object Types), their attributes (Property Types), the relationships among them (Link Types), and only subsequently models the operations that can be performed upon these structures (Action Types) Placeholder [1965, 1966]. This approach underpins the Web Ontology Language (OWL) Placeholder [1965], enterprise knowledge graphs, and industrial platforms such as Palantir Foundry Data Engine Placeholder

[1966]. It reflects a metaphysical commitment to the primacy of *continuants*—entities that exist in full at every moment of their existence and persist through changes in their attributes.

The object-first paradigm has proven effective in domains where the entity types are stable and well-understood: biomedicine (Gene Ontology), cultural heritage (CIDOC CRM), and enterprise data management (Palantir FDE). However, it encounters fundamental difficulties in domains characterized by high conceptual velocity, where new entity types emerge frequently and existing types evolve rapidly. Multi-agent systems powered by large language models (LLMs) exemplify such domains: autonomous agents generate hypotheses, propose concepts, validate one another’s contributions, and deprecate outdated ideas at a rate that exceeds human curation capacity Placeholder [1961, 1962]. In this setting, the object-first paradigm creates a bottleneck: every new concept type must be manually defined before instances can be created, and every status transition must be explicitly modeled as a permissible operation on an object state.

A second limitation of the object-first paradigm concerns the treatment of *state*. In conventional ontology platforms, a concept’s status (e.g., **proposed**, **validated**, **deprecated**) is stored as a mutable field. This field can diverge from the concept’s actual history when updates occur outside the governed action pathway—through race conditions between concurrent agents, manual overrides, or bugs in external systems. The resulting inconsistency is not merely a software bug; it is an ontological problem: the stored state no longer accurately represents the entity’s true ontological status, which is constituted by the history of events that have affected it.

1.2 Research Gap: Concept Lifecycle Governance as Ontological Analysis

The specific research gap addressed in this paper can be formulated as follows: *How can an operational ontology achieve lifecycle traceability and multi-agent governance without requiring object-first mutable state, expert curation, or heavyweight tooling?* This gap has both an engineering dimension and an ontological dimension.

The **engineering dimension** concerns deployment friction. Existing operational ontology platforms require specialized expertise, dedicated infrastructure, and manual curation workflows that do not scale to the velocity of agent-generated conceptual output Placeholder [1967, 1968]. LLM-based agents cannot participate in ontology construction because they lack programmatic access to the proprietary interfaces through which ontologies are defined.

The **ontological dimension** concerns the status–history relationship. In an object-first ontology, an entity’s status is a property of the entity. In an event-first ontology, an entity’s status is a derivative of its history. The transition from object-first to event-first is not merely a software design pattern (though it resembles event sourcing Placeholder [1979, 1980]); it is an ontological reorientation that changes the fundamental categories of the ontology. This reorientation has not been systematically analyzed in the applied ontology literature.

This paper addresses both dimensions by presenting ADL Lite, an event-first operational ontology in which concepts are modeled as append-only, cryptographically linked EventChains, and all lifecycle state is derived exclusively from event history. The paper’s primary contribution is an **ontological analysis** that situates ADL Lite within the landscape of foundational ontologies (BFO, DOLCE, UFO), formalizes its categories and identity conditions, and proves key properties of its derivation semantics.

1.3 Contributions

The contributions of this paper are organized around three axes: ontological analysis, formal semantics, and empirical validation.

Contribution 1: Ontological analysis of event-first concept governance. We analyze ADL Lite’s core categories through the lens of three influential foundational ontologies:

- We show that ADL Lite’s **Event** corresponds to the BFO category of *occurrent* and the DOLCE category of *perdurant*.
- We argue that ADL Lite’s **Concept** is best understood as a BFO *generically dependent continuant* or a DOLCE *non-physical object*.
- We demonstrate that ADL Lite’s L3 relations instantiate UFO *relators* with event-based lifecycles.
- We formalize identity conditions for events, concepts, and chains, and analyze three forms of ontological dependence (historical, generic, mutual).

Contribution 2: Formal derivation semantics with proved properties. We provide a complete formal foundation comprising:

- An event alphabet $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$ partitioned into lifecycle and communication events.
- A well-formedness predicate $\text{WF}(C)$ over EventChains.
- Status derivation $\delta(C)$ and confidence aggregation $\gamma(C)$ as deterministic functions.
- Fork and confluence semantics.
- Six proved properties: determinism (Theorem 1), confluence under fork (Theorem 2), transition monotonicity (Theorem 3), confidence boundedness (Theorem 4), confidence monotonicity under independent validation (Theorem 5), and status–confidence consistency (Theorem 6).
- An analysis of expressive power in relation to Event Calculus and Description Logic.

Contribution 3: Empirical validation of ontological consistency. We evaluate ADL Lite on 201 EventChains derived from the IBM Anti-Money Laundering (AML) HI-Small dataset (9,300 events). The evaluation confirms:

- Zero integrity failures across all chains, including the longest chain (300 events).
- 100% accuracy on 2,204 exhaustive status derivation test cases.
- Perfect precision and recall ($P = R = F1 = 1.0$) on 13 precondition enforcement test cases.
- Linear scalability: 31,100 events per second on modest hardware.

These results do not constitute domain-level evaluation of AML effectiveness; they validate that the event-first ontological architecture is computationally feasible, cryptographically sound, and empirically correct.

1.4 Paper Organization

Section 2 surveys related work in foundational ontologies, ontology engineering, and event-centric modeling. Section 3 presents the core ontological analysis: categories, identity conditions, ontological dependence, and alignment with BFO, DOLCE, and UFO. Section 4 describes the ADL Lite architecture, formal semantics, and comparison with Event Calculus and Description Logic. Section 5 reports empirical validation. Section 6 discusses implications and limitations. Section 7 concludes and identifies future work.

2 Related Work

2.1 Upper Ontologies and Foundational Ontologies

Foundational ontologies provide top-level categories that constrain and guide domain ontology development. Three influential frameworks—Basic Formal Ontology (BFO), the Descriptive Ontology for Linguistic and Cognitive Engineering (DOLCE), and the Unified Foundational Ontology (UFO)—offer distinct but complementary perspectives on the categories of entity, the nature of events, and the relationship between objects and processes. Understanding ADL Lite’s position with respect to these frameworks requires examining their treatment of *occurrences* (temporal entities), *continuants* (enduring entities), and the ontological status of information objects.

2.1.1 Basic Formal Ontology (BFO)

BFO is a top-level ontology developed primarily for the biomedical and life-science domains Arp et al. [2015]. Its core distinction is between *continuants* (entities that exist in full at every moment of their existence) and *occurrences* (entities that unfold in time and have temporal parts). Continuants include independent continuants (material objects), dependent continuants (qualities, roles, dispositions), and generically dependent continuants (information content entities). Occurrences include processes (events with temporal extent).

BFO’s treatment of events is particularly relevant to ADL Lite. In BFO 2.0, a *process* is an occurrence that has a temporal extent and at least one participant that is a continuant Arp et al. [2015]. Processes are not changes of state; they are entities in their own right. A validation process, for instance, is not merely the transition of a concept from **provisional** to **validated**; it is an occurrence with its own identity, participants (the validating agent, the concept being validated), and temporal boundaries.

ADL Lite’s **Event** corresponds closely to BFO’s *process*. Both are occurrences with temporal extent and participants. However, ADL Lite does not adopt BFO’s full continuant/occurrence hierarchy; instead, it collapses the distinction into a single event model in which all state changes are represented as events. This simplification is motivated by the target domain (multi-agent concept governance), where the primary concern is recording what happened, not classifying entities into a complex hierarchy of types.

A second BFO concept relevant to ADL Lite is the *generically dependent continuant* (GDC). A GDC depends on some continuant for its existence, but not on any specific continuant Arp et al. [2015]. A poem, for instance, is a GDC because it depends on some physical copy (a book, a memory) but not on any specific copy. ADL Lite’s “concept” is best understood as a GDC: it depends on its EventChain for existence, but not on any specific event within the chain.

2.1.2 DOLCE

DOLCE is a foundational ontology designed with a strong cognitive-linguistic orientation Gangemi et al. [2002]. Its top-level categories include *particulars* (entities that exist in space and time) and *universals* (entities that can be instantiated). Among particulars, DOLCE distinguishes *perdurants* (entities that are only partially present at any time) from *endurants* (entities that are wholly present at any time).

DOLCE’s category of *perdurant* corresponds to BFO’s *occurrence* and ADL Lite’s **Event**. However, DOLCE makes finer-grained distinctions among perdurants: *events* (changes of state), *processes* (activities with no definite endpoint), *states* (stable conditions), and *accomplishments* (activities with a definite endpoint) Gangemi et al. [2002]. ADL Lite’s event types do not explicitly adopt this

taxonomy, but they can be mapped to it: **REGISTER** and **VALIDATE** are events (changes of state), while **ANNOUNCE** and **PUBLISH** are accomplishments (activities with a definite endpoint).

DOLCE’s category of *non-physical object* (NPO) is also relevant. NPOs include social objects (e.g., organizations, contracts), information objects (e.g., documents, databases), and cognitive objects (e.g., thoughts, beliefs) Gangemi et al. [2002]. ADL Lite concepts are information objects that carry social weight (their status reflects community acceptance). In DOLCE terms, they are NPOs that depend on a community of agents for their interpretation and validation.

A key DOLCE principle is *descriptive adequacy*: the ontology should capture the categories that humans naturally use to conceptualize the world Gangemi et al. [2002]. ADL Lite’s document model (L1–L4) is designed with this principle in mind: the layers correspond to human cognitive distinctions between identity (L1), narrative (L2), assertions (L3), and actions (L4). This alignment is intentional: ADL Lite targets human-LLM collaborative authoring, so its categories must be cognitively accessible.

2.1.3 Unified Foundational Ontology (UFO)

UFO is a foundational ontology developed specifically for conceptual modeling and software engineering Guizzardi [2005]. It combines insights from BFO, DOLCE, and cognitive linguistics into a framework designed for building domain ontologies and conceptual models. UFO’s distinctive contribution is its treatment of *types*, *roles*, *phases*, and *mixins*—categories that are essential for modeling dynamic domains but are not explicitly represented in BFO or DOLCE.

UFO distinguishes three ontological levels: *individuals* (particular entities), *types* (categories that classify individuals), and *concepts* (mental representations of types) Guizzardi [2005]. ADL Lite operates primarily at the type level: `concept_id` identifies a type (e.g., “Capital Trap”), and the EventChain records the history of that type. However, ADL Lite also has elements of the individual level: each event is an individual occurrent, and each concept instance (e.g., a specific application of the Capital Trap pattern) is an individual.

UFO’s treatment of *events* and *actions* is particularly relevant. In UFO, an event is a change in the world, while an action is an intentional event performed by an agent to achieve a goal Guizzardi [2005]. ADL Lite’s event type system captures this distinction: **REGISTER**, **VALIDATE**, **DEPRECATE** are actions (intentional, performed by agents), while **RELATE**, **EVIDENCE**, **SEAL** are events (assertions that change relational structure). The precondition system in ADL Lite reflects the *plan* aspect of actions: an action is not merely a change; it is a change that is governed by norms.

UFO’s category of *relator* is also relevant to ADL Lite’s L3 relation blocks. A relator is an entity that mediates the relationship between two or more entities Guizzardi [2005]. In ADL Lite, a relation between two concepts is established by a **RELATE** event and can be revoked by a future event. This event-based lifecycle aligns with UFO’s view that relators are brought into existence and destroyed by events.

2.1.4 Synthesis: ADL Lite’s Position in the Foundational Ontology Landscape

Table 1 summarizes the comparison across the three foundational ontologies.

ADL Lite does not fully align with any single foundational ontology. Instead, it adopts a *pragmatic pluralism*: it uses BFO’s continuant/occurrent distinction as a conceptual guide, DOLCE’s descriptive adequacy as a design principle, and UFO’s event/action distinction as a modeling framework. This pluralism is deliberate: the target domain (multi-agent concept governance) requires a lightweight ontology that can be authored by non-experts in Markdown, not a heavyweight foundational ontology that requires expert curation in OWL.

Table 1: Comparison of foundational ontology perspectives on ADL Lite categories.

Feature		BFO	DOLCE	UFO
Event category		Process (occurrent)	Perdurant (event)	Event / Action
Concept category	cate-	Generically dependent continuant	Non-physical object (information)	Conceptual entity / Type
Relation category	cate-	Relational quality	Quality (relational)	Relator
Identity of concept	con-	concept_id (rigid designator)	Social object identity	Type identity
Temporal reasoning	reason-	Process parthood	Temporal part	Event sequence
Agent involvement	involve-	Process has participant	Agentive physical object	Agent performs action

The cost of this pluralism is that ADL Lite cannot directly import BFO, DOLCE, or UFO modules without translation. The benefit is that the ontology is optimized for its intended use case: lightweight, human-LLM collaborative, event-first concept governance. Section 3 provides the detailed ontological analysis that justifies these alignment choices.

2.2 Ontology Engineering and Knowledge Graph Construction

The Semantic Web vision articulated by Berners-Lee, Hendler, and Lassila Berners-Lee et al. [2001] established the foundational goal of machine-readable, interlinked knowledge representations. This vision was operationalized through OWL 2, whose formal semantics rooted in description logics provide expressiveness ranging from the lightweight OWL 2 EL profile to the fully decidable OWL 2 DL W3C OWL Working Group [2012]. Standard toolchains including Protégé, Apache Jena, and Virtuoso Universal Server enable ontology authoring, persistence, and SPARQL query evaluation at considerable scale. Reasoners such as HermiT and Pellet support automated classification and consistency checking, albeit with computational costs that scale poorly for large or rapidly evolving knowledge bases Placeholder [1966].

However, the deployment cost of full OWL-based pipelines remains prohibitive for many operational contexts. Triple stores require dedicated infrastructure; OWL reasoners impose batch-oriented computation ill-suited to streaming workloads; and the expertise required for ontology engineering creates a bottleneck distinct from application development Placeholder [1967]. These factors confine heavyweight Semantic Web technologies to domains where upfront investment in formal knowledge representation is justified by long-term reasoning requirements.

In response, the community has developed lightweight approaches that trade full logical expressiveness for accessibility. Schema.org provides a pragmatic, property-centric vocabulary adopted on over 40% of web pages Placeholder [1968]. JSON-LD extends conventional JSON with namespace support, allowing RDF-compatible serialization without requiring full Linked Data processing Sporny et al. [2020]. SHACL provides data validation without inference, enabling closed-world constraint checking over RDF graphs W3C [2017]. These technologies lower the barrier to structured data publishing but remain tethered to the RDF data model, excluding the vast corpus of domain knowledge encoded in Markdown documentation and plain-text notes.

The past decade has witnessed significant growth in Markdown-based knowledge management tools. Foam, Obsidian, and Dendron extend plain-text Markdown with wiki-style links, backlinks,

and graph visualization, enabling knowledge workers to construct personal knowledge graphs without leaving their editing environment Placeholder [1976]. Dendron introduces hierarchical note schemas with refactoring capabilities that preserve link integrity during structural changes Placeholder [1974]. These tools demonstrate that meaningful semantic structure can emerge from authoring conventions. However, they lack machine-checkable coordination primitives: no constraint language prevents schema drift, and no consensus mechanism mediates conflicting contributions from multiple agents. ADL Lite builds directly on this document-native paradigm while addressing each of these gaps.

2.3 Provenance, Trust, and Verifiable Publishing

The landscape of Semantic Web provenance encompasses several standards that address complementary aspects of the problem space that ADL Lite targets: recording *who* said *what*, *when*, and with *what* evidentiary basis.

W3C PROV-O. The W3C PROV Ontology W3C [2013] provides a standard model for provenance interchange, expressing the PROV Data Model in OWL 2. PROV-O defines core concepts including **Entity**, **Activity**, and **Agent**, along with properties such as **wasGeneratedBy** and **wasAssociatedWith** that establish causal and responsibility relationships. ADL Lite’s EventChain can be directly mapped to PROV-O: each Event corresponds to a **prov:Activity**, the concept being modified corresponds to a **prov:Entity**, and the **actor** field maps to **prov:wasAssociatedWith**. However, PROV-O alone does not provide cryptographic integrity guarantees—it is a data model for describing provenance, not a mechanism for verifying it. ADL Lite complements PROV-O by extending its descriptive provenance model with built-in SHA-256 hash chaining.

Nanopublications and Trusty URIs. The nanopublication framework Kuhn et al. [2015] provides a standardized way to publish small, granular scientific claims with cryptographically signed provenance. Each nanopublication consists of three named graphs: an assertion graph, a provenance graph, and a publication-info graph Carroll et al. [2005]. The Trusty URI mechanism Kuhn and Dumontier [2015] provides content-addressed identifiers via Base64-encoded SHA-256 hashes, ensuring that any modification to a nanopublication’s content changes its URI. This achieves decentralized verifiable publishing—a goal shared with ADL Lite’s append-only EventChain. Table 2 provides an empirical comparison across nine operational dimensions.

Linked Data Proofs. The W3C Verifiable Credential Data Integrity specification W3C Verifiable Credentials Working Group [2024] describes mechanisms for ensuring authenticity and integrity of digital documents through digital signatures over canonicalized RDF datasets. Underpinning this is the RDF Dataset Canonicalization algorithm (URDNA2015) W3C [2023], which produces a deterministic serialization regardless of original triple ordering. This technology provides authenticated provenance that ADL Lite currently lacks: the ability to bind actor identities to events through cryptographic signatures rather than string identifiers. ADL Lite’s Phase 3 includes LD-Proof integration for authenticated events.

RDF-star. RDF-star W3C RDF-star Working Group [2024] provides statement-level annotation by enabling triples to function as subjects or objects of other triples, without the syntactic verbosity of explicit reification. ADL Lite’s L3 relation blocks—encoding **source-relation-target** triples within Markdown fenced code blocks—can be viewed as a Markdown-native form of RDF triples, with each EventChain entry providing statement-level provenance analogous to an RDF-star annotation.

2.4 Event-Centric Ontologies

The ontologies surveyed above vary in their treatment of events. This subsection examines three event-centric approaches—CIDOC CRM, SEM/LODE, and RDF Stream Processing—that share

Table 2: Empirical comparison of nanopublications with Trusty URIs and ADL Lite EventChains.

Dimension	Nanopublications Trusty URIs	+ ADL Lite EventChain
Granularity	Atomic claim (single RDF triple or small graph)	Concept lifecycle (multi-event chain)
Mutability	Immutable: new version = new Trusty URI	Append-only: events accumulate, status derived
Authentication	RSA digital signatures per nanopub	String identifiers (Phase 1); Ed25519 LD-Proofs (Phase 3)
Integrity mechanism	Content-addressed SHA-256 hash in URI	Cryptographic hash chain (h_{prev} linkage)
Structural overhead	31–86% non-assertion triples	≈ 200 bytes/hash per event ($< 1\%$ for typical chains)
Infrastructure	RDF/SPARQL triplestore + nanopub server	Markdown files in standard Git repositories
Verification time	$O(1)$ per nanopub (hash comparison)	$O(n)$ per chain (sequential hash verification)
Lifecycle governance	None (status is publication metadata)	Deterministic status derivation + precondition rules
Fork support	Manual: create new nanopub, no automatic linkage	Automatic: FORK event + derived child chain

ADL Lite’s premise that *change in the world is best understood through events* rather than through direct property mutations of persistent objects.

CIDOC CRM. The CIDOC Conceptual Reference Model ICOM-CIDOC [2014] is an ISO 21127 standard for cultural heritage information integration. CIDOC CRM is fundamentally event-centric: it models reality through the **E5 Event** class that brings together **E39 Actor** participants with **E18 Physical Thing** objects in spatiotemporal contexts. The ontology’s core commitment is that change is understood through events, not through property mutations. ADL Lite shares this philosophical stance: concept lifecycle changes are explicit events in an append-only chain rather than mutable field updates. The key difference is purpose: CIDOC CRM is *descriptive* (documenting what happened), while ADL Lite is *operational* (enforcing what can happen through preconditions).

SEM and LODE. The Simple Event Model (SEM) van Hage et al. [2011] and Linked Open Descriptions of Events (LODE) Shaw et al. [2011] represent lightweight alternatives for publishing event data on the Semantic Web. SEM provides a minimal vocabulary comprising **sem:Event**, **sem:Actor**, **sem:Place**, and **sem:Time**. LODE extends SEM with Dublin Core compatibility and SKOS-based event type hierarchies. Both are *descriptive* ontologies for event publishing: they enable data providers to expose event descriptions in RDF for consumption and querying. They do not provide constraint mechanisms for governing which events can occur, nor do they offer integrity guarantees for event sequences. ADL Lite complements these frameworks with an *operational* layer: the EventChain is a tamper-evident, precondition-governed mechanism for controlling concept lifecycle transitions.

RDF Stream Processing. The W3C RDF Stream Processing Community Group defined models for continuous processing over timestamped RDF triples arriving in streams W3C RDF Stream Processing Community Group [2013]. Systems such as C-SPARQL Barbieri et al. [2009] and CQELS Le-Phuoc et al. [2011] extend SPARQL with windowing operators and continuous

query semantics, enabling real-time pattern detection over streaming RDF data. ADL Lite’s EventChain can be viewed as a *materialized* RDF stream: each event appends triples to a persistent, cryptographically linked chain, enabling retrospective provenance queries rather than real-time pattern detection.

Conflict-Free Replicated Data Types. CRDTs Shapiro et al. [2011] provide a theoretical foundation for reconciling concurrent updates without centralized coordination. Automerge Martin et al. [2020] is a JSON CRDT library that preserves all concurrent edits rather than choosing a single winner. Yjs Nicolai and Dadam [2021] is a widely deployed CRDT framework for real-time collaborative editing, optimized for shared text documents. Recent research has explored applying CRDTs to knowledge graph editing Placeholder [2019]. ADL Lite’s Git-native design already provides branch-based concurrent authoring; what CRDTs would add is automatic merge semantics for event chains, eliminating the last-write-wins limitation without sacrificing tamper evidence.

2.5 Positioning

Palantir Foundry represents the most prominent industrial embodiment of an operational ontology platform. Its Data Engine exposes four core ontological primitives: Object Type, Property, Link, and Action Palantir Technologies [2024]. This architecture unifies “semantic elements” (nouns) with “kinetic elements” (verbs), enabling AI agents to both query and act upon the ontology through governed interfaces Palantir Technologies [2024]. Despite its sophistication, Palantir exhibits three limitations: enterprise-scale deployment cost, proprietary ontology authoring tooling, and no design for LLM-native authorship. ADL Lite addresses each by offering pip-installable deployment, Markdown-native authoring, and explicit multi-agent concept consensus.

Table 3 positions ADL Lite against three reference approaches across seven salient dimensions.

ADL Lite occupies a deliberate middle ground. Palantir delivers industrial-strength operational semantics at deployment cost and accessibility barriers that exclude small teams and LLM-native workflows. OWL/SPARQL pipelines offer unmatched formal expressiveness yet require specialized expertise and infrastructure. Markdown-based tools democratize knowledge capture but provide no machine-checkable guarantees against schema drift or conflicting definitions.

The gap that ADL Lite fills is the intersection of four properties that no existing approach combines: (a) document-native authoring in plain Markdown, (b) built-in tamper detection through cryptographic hash chaining, (c) lifecycle state machines with declarative preconditions, and (d) pip-installable deployment requiring no enterprise infrastructure. PROV-O provides (a) only when combined with RDF authoring tools and lacks (b) natively; nanopublications provide (b) for published claims but lack (a), (c), and (d); CIDOC CRM provides event-centric modeling but requires heavyweight RDF infrastructure. ADL Lite enables multiple LLM agents proposing, challenging, and refining concepts within a shared document-native ontology, with the EventChain providing the audit trail and consensus mechanism that makes decentralized authorship accountable.

3 Ontological Analysis of ADL Lite

3.1 Philosophical Foundation: Event-First as Ontological Commitment

The architectural design of ADL Lite is grounded in an ontological inversion of conventional ontology engineering. Traditional operational ontologies—including OWL-based knowledge graphs, Palantir Foundry, and enterprise metadata platforms—adopt an *object-first* stance: an Object Type defines what entities exist, a Property Type describes their attributes, a Link Type connects them, and an Action Type is appended as an operational afterthought to model operations upon the previously

Table 3: Positioning of ADL Lite against reference approaches.

Dimension	Palantir FDE	OWL/SPARQL	Markdown- only	ADL Lite
Deployment Cost	Enterprise ($M+$)	High (triple store + rea- soner)	Low (Git repo)	Very Low (<code>pip install</code> + Git)
LLM Authorship	Not designed	Requires ontol- ogy expertise	Informal (free- text)	Native (YAML + Markdown)
Integrity Guar- antees	Platform- managed	Reasoner- based consis- tency	None	SHA-256 EventChain (append-only)
Action Precondi- tions	TypeScript functions	OWL/SWRL rules	None	Declarative YAML + SSA validation
Consensus Mech- anism	Centralized (admin UI)	N/A (single au- thority)	N/A (no coor- dination)	Multi-agent quorum + event history
Scalability	Billions of ob- jects	Millions of triples	Thousands of notes	Pilot: 201 chains; target: 10^4
Standards Align- ment	Proprietary	Full W3C stack	None	Partial (PROV-O mappable, SHACL- integrable)

defined object structures Placeholder [1966, 1974]. This ordering reflects a metaphysical commitment to the *primacy of continuants*: entities are conceived as self-subsistent bearers of properties that persist through changes in their attributes.

ADL Lite inverts this hierarchy. Drawing on Wittgenstein’s *Tractatus Logico-Philosophicus* §1.1—“*Die Welt ist die Gesamtheit der Tatsachen, nicht der Dinge*” (“The world is the totality of facts, not of things”)—the system treats the event as the fundamental ontological primitive Placeholder [1969, 1970]. On this view, an object has no ontological standing independent of the facts in which it appears. A concept’s status (**validated**, **deprecated**, **forked**) is not an intrinsic property of the concept but a relational property constituted by the history of events that have occurred in its lifecycle.

This philosophical commitment translates into three governing ontological principles:

- (1) **No implicit state changes.** Every mutation of ontological state is captured as an explicit event. There are no “hidden” updates, no side-effectful assignments, and no mutable fields that can diverge from their event histories.
- (2) **Events are occurrents.** Events are temporal entities that happen; they do not endure through time. An event has no spatial extent and no existence outside the interval at which it occurs. This aligns with the BFO category of *occurrent* Arp et al. [2015] and DOLCE’s category of *perdurant* Gangemi et al. [2002].
- (3) **All derived properties are historical.** Status, confidence, and validator identity are computed by deterministic functions over the event sequence. They are not stored; they are re-derived on demand. This eliminates an entire class of consistency bugs in which stored state diverges from logical history.

3.2 Categories and Taxonomy

ADL Lite’s category system can be analyzed through the lens of three influential upper ontologies: Basic Formal Ontology (BFO) Arp et al. [2015], Descriptive Ontology for Linguistic and Cognitive Engineering (DOLCE) Gangemi et al. [2002], and the Unified Foundational Ontology (UFO) Guizzardi [2005]. Table 4 presents the cross-mapping. The following subsections discuss each category in detail and justify alignment choices.

Table 4: Cross-mapping of ADL Lite categories to upper ontologies. Entries marked “—” indicate intentional non-alignment; entries marked “≈” indicate approximate correspondence.

ADL Lite	BFO 2.0	DOLCE	UFO	N
Event	<code>occurrent</code>	<code>perdurant</code>	<code>Event</code>	T
EventChain	<code>process</code>	<code>amount-of-matter≈</code>	<code>Complex Event</code>	Se
Concept	<code>generically dependent continuant</code>	<code>non-physical object≈</code>	<code>Conceptual Entity</code>	D
Relation (L3)	<code>relational quality</code>	<code>quality</code>	<code>Relator</code>	M
Action (L4)	<code>planned process</code>	<code>action</code>	<code>Action</code>	In
Actor	<code>agent</code>	<code>physical agent</code>	<code>Agent</code>	C
Payload	<code>information content entity</code>	<code>information object</code>	<code>Information Entity</code>	S
Hash	<code>generically dependent continuant</code>	—	—	C

3.2.1 Event as Occurrent/Perdurant

In BFO, an *occurrent* is an entity that unfolds itself in time. Occurrents include processes (e.g., a cell division), and they are ontologically distinct from *continuants* (e.g., a cell), which exist in full at every moment of their existence Arp et al. [2015]. In DOLCE, the corresponding category is *perdurant*: an entity that is only partially present at any time, in the sense that some of its temporal parts may be absent Gangemi et al. [2002].

ADL Lite’s **Event** is a perdurant in the DOLCE sense and an occurrent in the BFO sense. It is a temporal entity with a definite beginning and end (the timestamp). It has no spatial location (unless the payload carries one, which is domain-specific). It does not endure: once a **VALIDATE** event has occurred, it is complete; it does not continue to exist as a present entity. What endures is the *record* of the event (the serialized JSON/YAML in the Markdown file), but this record is an *information content entity*, not the event itself.

In UFO, events are also treated as perdurants, but UFO additionally distinguishes *events* (changes in the world) from *actions* (intentional events performed by agents) Guizzardi [2005]. ADL Lite’s event type system captures this distinction: **REGISTER**, **VALIDATE**, **DEPRECATE**, **FORK**, **ARCHIVE** are actions (intentional, performed by an actor), while **RELATE**, **EVIDENCE**, **SEAL** are events (assertions that change the relational structure of the concept graph).

3.2.2 Concept as Generically Dependent Continuant

The ontological status of a “concept” in ADL Lite is subtle. A concept is not a physical object (it has no mass, no spatial location). It is not a mental object (it exists outside any individual mind, in the shared Markdown file). It is not a universal (it is not instantiated by particulars). We propose that an ADL Lite concept is best understood as a *generically dependent continuant* in BFO terms Arp et al. [2015]: an entity that depends on some continuant or other for its existence, but not on any *specific* continuant.

A concept depends generically on its EventChain: if the chain were destroyed, the concept would cease to exist. But the concept does not depend on any specific event within the chain—events can be added and the concept persists. The concept’s identity is determined by the *genesis event* (the first event in the chain, with its `concept_id`), not by the totality of events. This is analogous to how a BFO *generically dependent continuant* such as a poem depends on some physical copy for its existence, but not on any specific copy.

In DOLCE, the closest category is *non-physical object* (NPO), which includes social objects, information objects, and cognitive objects Gangemi et al. [2002]. ADL Lite concepts are information objects that carry social/institutional weight (their status determines whether they are accepted by the community). In UFO, concepts are *conceptual entities* or *types* in the conceptual space of the community Guizzardi [2005].

3.2.3 EventChain as Process

An EventChain is a sequence of events. In BFO, it is best understood as a *process*: a complex occurrent that has temporal parts (the individual events) and unfolds over time Arp et al. [2015]. The chain is not a continuant because it does not exist in full at any single moment of time—at time t , only the events up to t have occurred.

This has implications for identity conditions. In BFO, a process is individuated by its constituent processes and their temporal relations. An EventChain is individuated by the ordered sequence of its events and their cryptographic linkage. Two chains with the same events in the same order are the same chain (modulo the `concept_id`, which anchors the chain to a specific concept).

3.2.4 Relation as Relational Quality / Relator

ADL Lite’s L3 relation blocks (`adl:relation`) assert typed edges between concepts: `source -relation-> target`. In BFO, this is best analyzed as a *relational quality*: a quality that inheres in one entity and relates it to another Arp et al. [2015]. For example, the relation “isomorphic-to” between two concepts is a relational quality that inheres in the source concept and points to the target concept.

In UFO, the corresponding category is *relator*: an entity that mediates the relationship between two or more entities Guizzardi [2005]. A relator exists because the related entities exist, and it is destroyed when the relationship ceases to hold. In ADL Lite, a relation is established by a `RELATE` event and can be revoked (in future work) by a `UNRELATE` event. This event-based lifecycle aligns with the UFO notion that relators are brought into existence and destroyed by events.

DOLCE’s category of *quality* is broader and includes both intrinsic qualities (e.g., color) and relational qualities (e.g., being taller than) Gangemi et al. [2002]. ADL Lite relations are relational qualities in the DOLCE sense.

3.2.5 Action as Planned Process / Intentional Event

ADL Lite’s L4 action types (`REGISTER`, `VALIDATE`, etc.) are actions in the full philosophical sense: they are intentional events performed by agents for reasons. In BFO, they are *planned processes*: processes that realize a plan Arp et al. [2015]. In UFO, they are *actions*: intentional events performed by agents to achieve a goal Guizzardi [2005].

The precondition system in ADL Lite captures the *plan* aspect: an action can only be performed if certain conditions hold (e.g., `VALIDATE` requires `status == provisional`). This is not merely a computational check; it is an ontological commitment to the idea that actions are governed by norms that constrain which events can occur in which states.

3.3 Identity Conditions

Identity conditions specify when two entities are the same. ADL Lite requires identity conditions for three categories: events, concepts, and chains.

3.3.1 Event Identity

An event e is identical to an event e' if and only if:

$$e = e' \iff e.\text{event_id} = e'.\text{event_id} \quad (1)$$

The `event_id` is a UUID generated at event creation time. It is independent of the event’s content, timestamp, or chain position. This means that two events with identical content but different `event_ids` are *distinct events* (they occurred at different times or in different contexts). This aligns with the intuition that two validation events by the same actor at different times are different events, even if their payloads are identical.

The cryptographic hash $e.\text{hash}$ is *not* an identity criterion; it is an integrity criterion. Two events with the same hash have the same content and linkage, but they may still be different events (if they have different `event_ids`, which is impossible because the `event_id` is in the hash input). In practice, the hash uniquely determines the event’s content, but the `event_id` is the canonical identifier.

3.3.2 Concept Identity

A concept c is identical to a concept c' if and only if:

$$c = c' \iff c.\text{concept_id} = c'.\text{concept_id} \quad (2)$$

The `concept_id` is a human-readable identifier (e.g., `disc-capital-trap`) assigned at registration time. It is independent of the concept's status, confidence, or event history. This means that a concept retains its identity even if all its events are forked, deprecated, or archived.

This choice reflects an ontological commitment to *historical identity*: a concept is the same concept throughout its lifecycle because it has the same genesis event (the `REGISTER` event that created it). The `concept_id` is a rigid designator for the concept, analogous to how a proper name rigidly designates an individual in modal logic.

A forked concept (e.g., `disc-capital-trap-v2`) is a *different concept* because it has a different `concept_id`, even though it is derived from the original. This is analogous to how a biological daughter is a different organism from her mother, even though she derives from her mother's genetic material.

3.3.3 EventChain Identity

An EventChain C is identical to an EventChain C' if and only if:

$$C = C' \iff C.\text{concept_id} = C'.\text{concept_id} \wedge \text{events}(C) = \text{events}(C') \quad (3)$$

Two chains are the same if they have the same `concept_id` and the same ordered sequence of events. The cryptographic linkage ensures that the event sequence is ordered, so two chains with the same events in different orders are different chains.

3.4 Ontological Dependence

Ontological dependence concerns which entities depend on which others for their existence. ADL Lite exhibits three forms of dependence:

3.4.1 Historical Dependence of Concept on EventChain

A concept c *historically depends* on its EventChain C : c would not exist if C had not been created. This is a form of *rigid existential dependence* Fine [1995]: the concept cannot exist without the chain. The dependence is historical because the concept's identity is determined by the genesis event in the chain.

This dependence is asymmetric: the chain can exist without the concept (in the trivial sense that the chain is the concept's history). But the concept cannot exist without the chain because the concept *is* its history.

3.4.2 Generic Dependence of Status on Events

A concept's status s (e.g., `validated`) *generically depends* on the events in its chain: s depends on there being at least one `VALIDATE` event, but not on any *specific* `VALIDATE` event. If a different agent had performed the validation at a different time, the status would still be `validated`. This is analogous to how a BFO generically dependent continuant depends on some bearer but not on any specific bearer.

3.4.3 Mutual Dependence of Relations on Concepts

A relation r between concepts c_1 and c_2 *mutually depends* on both concepts: r cannot exist if either c_1 or c_2 ceases to exist. This is a form of *mutual rigid dependence* Fine [1995]. If `concept-A` is deprecated and removed from the corpus, the relation “`concept-A` isomorphic-to `concept-B`” ceases to hold, even if `concept-B` still exists.

3.5 Comparison with Upper Ontologies: Alignment Choices and Deviations

Table 4 reveals three intentional deviations from standard upper-ontology practice:

Deviation 1: No distinction between continuants and occurrents at the language level. In BFO and DOLCE, the distinction between continuants and occurrents is a *language-level* distinction: the ontology language (OWL, FOL) enforces that continuants cannot be subclasses of occurrents. ADL Lite does not enforce this distinction at the language level; it is a *conceptual* distinction that is enforced by the event model. A “concept” is treated as a continuant in some contexts (it has an identity that persists through time) and as an occurrent in others (its state is computed from events). This flexibility is deliberate: it allows the same entity to be viewed under different ontological descriptions without requiring multiple representations.

Deviation 2: No explicit quality hierarchy. BFO and DOLCE both have elaborate hierarchies of qualities (intrinsic, relational, qualitative, etc.). ADL Lite does not model qualities explicitly; instead, qualities are encoded as payload fields in events. For example, “confidence” is not a quality that inheres in the concept; it is a value carried by `VALIDATE` events. This simplifies the ontology but loses the ability to reason about qualities independently of events.

Deviation 3: Actions as first-class citizens, not processes. In BFO, actions are a subclass of processes. In ADL Lite, actions are *co-equal* with objects at the language level: the action registry is defined before the object hierarchy. This reflects the event-first stance: actions are not operations *on* objects; they are the fundamental units of ontological reality, and objects are projections of event patterns.

These deviations are not arbitrary simplifications; they are motivated by the target application domain (multi-agent concept governance). In a domain where agents create, validate, and deprecate concepts at high velocity, the action-centric view provides a more natural modeling framework than the object-centric view. The cost is that ADL Lite cannot directly import BFO or DOLCE modules without translation; the benefit is that the ontology is optimized for its intended use case.

4 Architecture and Formal Semantics

4.1 Document Model: L1–L4 as Conceptual Modeling Layers

ADL Lite organizes concept documents into four semantic layers, each with distinct syntax, role, and corresponding event types. The layering separates identity metadata (L1), narrative content (L2), typed assertions (L3), and executable actions (L4), enabling both human readability and machine processing within a single Markdown file. Table 5 summarizes the document model.

The L1 layer comprises YAML front matter containing identity metadata: `adl_type`, `adl_id`, `status`, `confidence`, `novelty`, `scope`, and `provisional_names`. Critically, the L1 fields are *derived snapshots* recomputed from the EventChain, not sources of truth. A discrepancy between L1 `status` and the chain-derived status indicates an incomplete event history in the source document.

The L2 layer consists of the Markdown body: human- and LLM-readable prose. The Singular Subjectivity Assertion (SSA) prohibits fuzzy referents—including pronouns such as “this,” “that,” “it”—that would compromise cross-agent alignment Placeholder [1976]. SSA is configurable: `strict`

Table 5: The L1–L4 Document Model. Each layer contributes distinct semantics to the concept document.

Layer	Syntax	Role	Event Type
L1	YAML front matter	Identity metadata (derived snapshot)	SNAPSHOT
L2	Markdown body	Human/LLM narrative prose	—
L3	<code>adl:relation</code> , <code>adl:evidence</code> , <code>adl:seal</code>	Typed semantic assertions	RELATE, EVIDENCE
L4	<code>adl:action</code>	Typed actions with preconditions	REGISTER, VALIDATE

mode enforces explicit reference for all agent-authored content, while `lax` mode permits natural pronoun use for human-authored narrative.

The L3 layer introduces typed semantic assertions through fenced code blocks: `adl:relation` for typed edges between concepts (supporting predicates such as `isomorphic-to` and `specialisation-of`), `adl:evidence` for evidentiary support, and `adl:seal` for formal assertions. These blocks are machine-parseable RDF-like triples that remain editable in ordinary text editors.

The L4 layer introduces `adl:action` blocks: typed actions with structured precondition rules that govern lifecycle transitions and trigger side effects. The four-layer design enables a single Markdown file to serve simultaneously as human-readable documentation, LLM authoring surface, machine-parseable semantic assertion container, and auditable action log.

4.2 EventChain: Core Data Structure

The EventChain is the fundamental data structure of ADL Lite. Each ontological concept is represented by exactly one EventChain: an append-only sequence of cryptographically linked events.

An *event* e is a structured record comprising nine fields: `event_id` (UUID-derived unique identifier), `concept_id` (the concept to which the event belongs), `event_type` (drawn from a closed set of lifecycle, assertion, and communication types), `actor` (the entity causing the event), `reasoning` (justification), `timestamp` (ISO-8601 occurrence time), `payload` (event-specific key-value data), `previous_event_id` (cryptographic link to the preceding event), and `hash` (SHA-256 digest of serialized content).

The event type system is partitioned into two disjoint sets:

$$\Sigma_{\text{life}} = \{\text{REGISTER, VALIDATE, DEPRECATE, FORK, ARCHIVE}\} \quad (4)$$

$$\Sigma_{\text{comm}} = \{\text{RELATE, EVIDENCE, SEAL, ANNOUNCE, PUBLISH, SNAPSHOT}\} \quad (5)$$

Lifecycle events affect concept status; communication events do not. This partition is central to the derivation semantics (Section 4.6).

4.3 Canonical Serialization and Cryptographic Integrity

Each event’s hash is computed over a canonical JSON serialization with the following properties: (i) keys are sorted lexicographically; (ii) floating-point values are rounded to six decimal places; (iii) all fields including the timestamp are included in the serialization; (iv) the hash field itself is excluded to avoid circular self-reference.

Timestamps are *included* in the hash input to ensure that two otherwise-identical events created at different times produce different hashes, preserving tamper-evidence for provenance. Event *ordering* within a chain uses cryptographic linkage (`previous_event_id`) rather than timestamp comparison, so clock skew does not affect chain validity.

The integrity verification predicate $\text{VerifyIntegrity}(C)$ checks three conditions for a chain $C = [e_1, \dots, e_n]$:

- (1) Genesis anchoring: $e_1.\text{previous_event_id} = \text{null}$ and $e_1.\text{prev_hash} = \text{genesis}$.
- (2) Cryptographic linkage: for all $i \in \{2, \dots, n\}$, $e_i.\text{prev_hash} = e_{i-1}.\text{hash}$.
- (3) Hash correctness: for all $i \in \{1, \dots, n\}$, $e_i.\text{hash} = \text{SHA-256}(\text{Canon}(e_i))$.

4.4 Action Executor with Structured Preconditions

The ActionExecutor validates L4 action blocks against a closed action registry defined in `adl_core_ontology.yaml`. Each action definition specifies preconditions as a list of **PreconditionRule** objects, each comprising a **field** (the ADLFrontMatter field to inspect), a **comparator** (EQ, NEQ, GT, GTE, LT, LTE, IN, EXISTS), and a **value** (the expected value or threshold).

Precondition evaluation uses explicit lambda dispatch on the **Comparator** enumeration; there is no runtime `eval()` or code injection path. This closed evaluation model achieves perfect precision and recall on all registered actions (Section 5.4).

4.5 Consensus and Status Transitions

Status transitions are governed by a transition matrix that defines which lifecycle events are permitted from each status. The matrix is total: every status has at least one permissible outgoing transition. The ConsensusEngine enforces these transitions by validating preconditions before appending events to chains.

Fork semantics deserve special attention. When a concept forks, a **FORK** event is appended to the original chain, and a new chain is created with a fresh **REGISTER** event. The original concept's status becomes **forked**; the new concept's status is **provisional**. Both chains evolve independently thereafter. This preserves the historical integrity of the original concept while enabling divergent evolution.

4.6 Formal Derivation Semantics

This section provides the complete formal foundation for the derivation semantics. All definitions are self-contained.

4.6.1 Event Alphabet and Well-Formedness

Let $\Sigma = \Sigma_{\text{life}} \cup \Sigma_{\text{comm}}$ be the event alphabet. An event is a tuple $e = (\tau, a, t, p, h, h_{\text{prev}}, e_{\text{prev}})$ where $\tau \in \Sigma$, $a \in \mathcal{A}$ (actor set), $t \in \mathbb{R}_{\geq 0}$ (timestamp), $p \in \mathcal{P}$ (payload), $h \in \{0, 1\}^{256}$ (hash), $h_{\text{prev}} \in \{0, 1\}^{256} \cup \{\text{genesis}\}$, and $e_{\text{prev}} \in \text{ID} \cup \{\text{null}\}$.

A chain $C = [e_1, \dots, e_n]$ is well-formed, denoted $\text{WF}(C)$, if:

- (1) $e_1.h_{\text{prev}} = \text{genesis}$ and $e_1.e_{\text{prev}} = \text{null}$.
- (2) All events in C share the same `concept_id`.
- (3) For all $i \geq 2$: $e_i.h_{\text{prev}} = e_{i-1}.h$ and $e_i.e_{\text{prev}} = e_{i-1}.\text{event_id}$.
- (4) For all i : $e_i.h = \text{SHA-256}(\text{Canon}(e_i))$.
- (5) All `event_id` values in C are pairwise distinct.

The canonicalization function $\text{Canon}(e)$ serializes e as UTF-8 JSON with sorted keys, six-decimal float rounding, and all fields (including timestamp) included.

4.6.2 Status Derivation δ

The status space is $S = \{\text{provisional}, \text{validated}, \text{deprecated}, \text{forked}, \text{archived}\}$. For a chain C , let $C_{\text{life}} = [e \in C \mid e.\tau \in \Sigma_{\text{life}}]$ be the lifecycle subsequence. Then:

$$\delta(C) = \begin{cases} \text{provisional} & \text{if } C_{\text{life}} = [] \\ f(\tau_{\text{last}}) & \text{otherwise} \end{cases} \quad (6)$$

where τ_{last} is the event type of the last element of C_{life} , and f maps REGISTER $\rightarrow$ provisional, VALIDATE $\rightarrow$ validated, DEPRECATE $\rightarrow$ deprecated, FORK $\rightarrow$ forked, ARCHIVE $\rightarrow$ archived.

4.6.3 Confidence Aggregation γ

Let $V = [e \in C \mid e.\tau = \text{VALIDATE}]$. Then:

$$\gamma(C) = \begin{cases} 0.0 & \text{if } V = [] \\ \min(1.0, c_{\text{base}} + 0.05 \times (N_{\text{vals}} - 1)) & \text{otherwise} \end{cases} \quad (7)$$

where $\text{actors}(V) = \{e.a \mid e \in V\}$, $\phi(a, V) = \max\{e.p.\text{confidence} \mid e \in V \wedge e.a = a\}$, $c_{\text{base}} = \max(0.5, \text{mean}_{a \in \text{actors}(V)} \phi(a, V))$, and $N_{\text{vals}} = |\text{actors}(V)|$.

The per-actor maxima ensure duplicate validations from the same actor do not compound. The base confidence floor of 0.5 ensures that any validated concept has at least moderate confidence. The bonus of 0.05 per additional distinct validator rewards independent confirmation.

4.6.4 Fork and Confluence Semantics

A fork transforms a concept C into (C_{fork}, C') where $C_{\text{fork}} = C \oplus [e_{\text{FORK}}]$ and $C' = [e'_{\text{REG}}]$ is a new chain with fresh identifier. The forked concept identifier is recorded in $e_{\text{FORK}}.p.\text{fork_target}$.

Theorem 1 (Determinism). For any well-formed C , $\delta(C)$ is unique and depends only on the last lifecycle event in C .

Proof. By definition, δ filters C to C_{life} (deterministic), checks emptiness (deterministic), and applies total function f to the last element (unique for ordered sequences). $\square$

Theorem 2 (Confluence under Fork). Let C fork to (C_{fork}, C') . Then $\delta(C_{\text{fork}}) = \text{forked}$ and $\delta(C') = \text{provisional}$.

Proof. $e_{\text{FORK}} \in \Sigma_{\text{life}}$ is the last lifecycle event in C_{fork} , so $f(\text{FORK}) = \text{forked}$. C' contains only e'_{REG} , so $f(\text{REGISTER}) = \text{provisional}$. $\square$

Theorem 3 (Transition Monotonicity). For any well-formed C and event e_{new} , the transition from $s = \delta(C)$ to $s' = \delta(C \oplus [e_{\text{new}}])$ is valid iff $e_{\text{new}}.\tau \in \Sigma_{\text{life}}$.

Proof. If $e_{\text{new}}.\tau \notin \Sigma_{\text{life}}$, then C_{life} is unchanged, so $s' = s$ (identity transition, always valid). If $e_{\text{new}}.\tau \in \Sigma_{\text{life}}$, then $s' = f(e_{\text{new}}.\tau)$, which is exactly the target state defined by the transition matrix. $\square$

Theorem 4 (Confidence Boundedness). For any well-formed C , $\gamma(C) \in [0, 1]$.

Proof. If $V = []$, $\gamma(C) = 0.0$. Otherwise, each $\phi(a, V) \in [0, 1]$ by payload validation, so their mean is in $[0, 1]$. Thus $c_{\text{base}} \geq 0.5$. The bonus term is non-negative, and the outer $\min(1.0, \cdot)$ caps at 1.0. $\square$

Theorem 5 (Confidence Monotonicity). Let $\gamma(C) = c$ and let e_{new} be a **VALIDATE** event from a new actor. Then $\gamma(C \oplus [e_{\text{new}}]) \geq c$, with strict inequality if $c < 1.0$.

Proof. Let $N'_{\text{vals}} = N_{\text{vals}} + 1$. The new base confidence c'_{base} is a weighted mean of the old mean and the new actor's confidence. Even if $c'_{\text{base}} \leq c_{\text{base}}$, the validator count increment adds 0.05 to the bonus term, ensuring the final value is at least c . If $c < 1.0$, the $\min(1.0, \cdot)$ is not binding, so strict inequality holds. $\square$

Theorem 6 (Status–Confidence Consistency). If $\delta(C) = \text{validated}$, then $\gamma(C) \geq 0.5$.

Proof. $\delta(C) = \text{validated}$ implies the last lifecycle event is **VALIDATE**, so $V \neq \emptyset$. Thus $c_{\text{base}} = \max(0.5, \text{mean}(\cdot)) \geq 0.5$. By Theorem 4, the final value is at least $c_{\text{base}} \geq 0.5$. $\square$

4.7 Comparison with Formal Event Frameworks

ADL Lite's event model can be compared with three classical formal frameworks for reasoning about events and change: Event Calculus, Situation Calculus, and Description Logic.

4.7.1 Event Calculus

The Event Calculus (EC), introduced by Kowalski and Sergot [1986], is a logic programming framework for reasoning about events and their effects. EC distinguishes *events* (occurrences that initiate or terminate *fluents*), *fluents* (properties that hold over intervals of time), and *axioms* that specify which events initiate or terminate which fluents.

In EC, a concept's status would be modeled as a fluent: $\text{Validated}(c)$ holds over an interval $[t_1, t_2]$ if a **VALIDATE** event initiated it at t_1 and no subsequent **DEPRECATE** event terminated it before t_2 . ADL Lite's $\delta(C)$ function computes exactly this fluent, but it does so by examining the entire event history rather than maintaining a set of currently holding fluents.

The key difference is temporal representation. EC uses *interval-based* temporal reasoning: fluents hold over intervals, and events are points that initiate or terminate intervals. ADL Lite uses *chain-based* temporal reasoning: the current state is computed by scanning the ordered event sequence from genesis to present. Both approaches are equivalent in expressive power for the class of properties that ADL Lite models (status transitions with no continuous change), but the chain-based approach is computationally simpler— $\delta(C)$ is $O(n)$ in the chain length—and provides built-in cryptographic integrity.

EC also supports *continuous change* (e.g., a moving object's position changes continuously over time). ADL Lite does not support continuous change; all state changes are discrete events. This is a deliberate limitation: the target domain (concept lifecycle governance) involves discrete state transitions (proposed $\rightarrow$ validated $\rightarrow$ deprecated), not continuous physical processes.

4.7.2 Situation Calculus

The Situation Calculus (SC), developed by McCarthy and Hayes [1969], is a first-order logical framework for reasoning about action and change. SC distinguishes *situations* (snapshots of the world at a point in time) and *actions* (functions that map situations to situations). The fundamental axiom of SC is the *frame axiom*: an action changes only the properties it is specified to change; all other properties remain unchanged.

ADL Lite's EventChain can be viewed as a sequence of situations: each event e_i transforms the situation S_{i-1} into S_i . The status derivation $\delta(C)$ computes the value of a specific fluent (status) in the final situation S_n . However, ADL Lite does not explicitly represent situations; it represents only the event sequence. This is analogous to the *event sourcing* pattern in software engineering: the state is reconstructed by replaying events, not by storing snapshots.

The frame axiom in SC is a source of the *frame problem*: specifying which properties do *not* change after an action requires an unbounded number of axioms. ADL Lite avoids the frame problem by construction: because there are no stored mutable fields, there are no properties that need to be preserved across state transitions. All derived properties are recomputed from the event sequence, so the “frame” is implicit in the derivation functions.

4.7.3 Description Logic Expressivity

ADL Lite’s derivation functions can be analyzed in terms of Description Logic (DL) expressivity. The status derivation $\delta(C)$ is a function from a sequence of events to a status value. In DL terms, this is a *role chain* query: the status of a concept is determined by the type of the last event in the chain of events related to the concept by the `previous_event_id` role.

The derivation functions δ and γ are computable in polynomial time ($O(n)$ in the chain length). They do not require the full expressive power of OWL-DL (which is NExpTime-complete for satisfiability). In fact, δ and γ can be expressed in the DL-lite family of lightweight description logics, which support efficient query answering over large datasets Calvanese et al. [2007].

Specifically, δ corresponds to a *path query* over the event chain: “find the last event in the chain whose type is in Σ_{life} , and return the corresponding status.” This query can be expressed in DL-Lite_R using role inclusion axioms. The confidence aggregation γ involves aggregation (mean, max) over sets of events, which goes beyond standard DL query answering but can be handled by DL extensions with aggregation operators Calvanese et al. [2007].

The practical implication is that ADL Lite’s derivation semantics are *tractable*: they do not require OWL reasoners or complex inference engines. This aligns with the design goal of lightweight deployment: the ontology can be processed by a simple Python script operating over Markdown files, without requiring a triple store or DL reasoner.

5 Empirical Validation

5.1 Validation Strategy: Correctness as Ontological Consistency

The empirical evaluation of ADL Lite is designed to validate three ontological hypotheses:

- (1) **Integrity hypothesis.** The cryptographic hash chain correctly detects all structural modifications (content tampering, reordering, deletion).
- (2) **Derivation hypothesis.** The status and confidence derivation functions produce correct results for all possible event sequences.
- (3) **Precondition hypothesis.** The action precondition system correctly accepts valid actions and rejects invalid ones.

These hypotheses are validated through four experiments: chain integrity (E1), status derivation exhaustiveness (E2), precondition enforcement (E4), and scalability on real-world data (E6). The experiments do not constitute domain-level evaluation of anti-money-laundering (AML) effectiveness; they validate that the event-first ontological architecture is computationally feasible and empirically correct.

5.2 Chain Integrity and Tamper Detection (E1)

E1 tests the integrity verification predicate `VerifyIntegrity(C)` on 60 chains: 50 valid chains and 10 intentionally corrupted chains. The corrupted chains exhibit four attack classes:

- **Content tampering:** Modify the payload of an existing event.
- **Event reordering:** Swap two events in the chain.
- **Event deletion:** Remove a middle event.
- **Event replay:** Copy an event from one chain into another.

Results: precision = 1.0, recall = 1.0, F1 = 1.0. All 50 valid chains pass verification; all 10 corrupted chains fail verification. The attack detection is deterministic: any modification to an event’s content, reordering of events, or deletion of events breaks the cryptographic linkage at the point of first alteration.

5.3 Status Derivation Exhaustiveness (E2)

E2 tests the status derivation function $\delta(C)$ on all possible event sequences of length up to 3 over the 5 lifecycle event types, plus 7 edge cases (empty chain, single-event chains, full lifecycle sequences, mixed lifecycle/communication sequences). The total test corpus comprises 2,204 cases.

Results: 2,204/2,204 correct (accuracy = 1.0). Edge cases confirm that communication events (Σ_{comm}) never modify status, while lifecycle events (Σ_{life}) deterministically drive transitions according to the transition matrix. This validates Theorem 1 (Determinism) and Theorem 3 (Transition Monotonicity) empirically.

5.4 Precondition Enforcement (E4)

E4 tests the ActionExecutor’s precondition validation on all 9 registered actions across 13 test cases covering valid preconditions, violated preconditions, boundary values, and type mismatches.

Results: precision = 1.0, recall = 1.0, F1 = 1.0. Zero false positives (no valid action rejected) and zero false negatives (no invalid action accepted). The closed Comparator enumeration eliminates runtime code injection; the absence of `eval()` ensures that precondition evaluation is decidable and safe.

5.5 Scalability on Real-World Data (E6)

E6 evaluates throughput and integrity on the IBM Anti-Money Laundering (AML) HI-Small dataset, comprising 9,300 transactions from 201 accounts. Each account is imported as an EventChain; transactions become `Event` objects. The mean chain length is 46.3 events; the maximum is 300 events.

Results: All 201 chains maintain integrity (zero failures). Wall-clock runtime is 299 ms, yielding throughput of approximately 31,100 events per second. Profiling indicates that the bottleneck is CSV parsing and event materialization, not chain operations or SHA-256 hashing.

Architectural contribution, not domain validation. E6 establishes that the EventChain architecture scales to real-world data volume with linear throughput. This is an architectural stress test, not a domain evaluation of AML effectiveness. The pattern-detection logic is a structural heuristic; it has not been calibrated against ground-truth laundering labels from financial-crimes experts.

5.6 Summary of Results

Table 6 summarizes the empirical validation. All four experiments achieve perfect scores, providing strong evidence for the three ontological hypotheses. The experiments validate that the event-first architecture is computationally feasible and empirically correct; they do not, by themselves, demonstrate the novelty of the event-first ontological stance. That novelty is established through the ontological analysis (Section 3) and formal semantics (Section 4.6).

Table 6: Summary of empirical validation results.

Experiment	Hypothesis	Metric	Result	Scale
E1	Integrity	P / R / F1	1.0 / 1.0 / 1.0	60 chains
E2	Derivation	Accuracy	1.0 (2,204/2,204)	Exhaustive to length 3
E4	Precondition	P / R / F1	1.0 / 1.0 / 1.0	13 test cases
E6	Scalability	Integrity	0 failures	201 chains, 9,300 events

6 Discussion

6.1 Event-First Architecture: Ontological Implications

The empirical results provide support for the event-first design, but the deeper contribution lies in the ontological analysis. By treating events as the fundamental primitive, ADL Lite inverts the traditional object-first hierarchy and raises three ontological questions that merit discussion.

Question 1: What is the ontological status of a concept? In an object-first ontology, a concept is a continuant: it exists in full at every moment, and its properties change over time. In ADL Lite, a concept is a generically dependent continuant that depends on its EventChain for existence. This raises the question: does the concept exist before any events have occurred? The answer is no: a concept comes into existence at the moment of its genesis event (**REGISTER**). Prior to that event, there is no concept—only the potential for one. This is analogous to how, in BFO, a process comes into existence at the moment it begins: there is no “pre-existing” process waiting to unfold.

Question 2: Are status and confidence real properties or derived properties? In an object-first ontology, status is a real property that inheres in the concept. In ADL Lite, status is a derived property: it is computed by $\delta(C)$ and has no existence independent of the events that constitute the chain. This aligns with the DOLCE view that some properties are “constructed” by cognitive agents rather than discovered in the world Gangemi et al. [2002]. A concept’s status is not a feature of reality; it is a feature of the community’s judgment about the concept, as recorded in the event history.

Question 3: What happens to a concept when it is deprecated? In an object-first ontology, deprecation might be modeled as a change of a status field. In ADL Lite, deprecation is the appendage of a **DEPRECATE** event to the chain. The concept continues to exist (it retains its `concept_id`), but its status changes to `deprecated`. The historical events remain intact, providing a complete audit trail of why the concept was deprecated and by whom. This preserves the “right to be forgotten” in reverse: the concept is not erased, but it is explicitly marked as no longer endorsed.

6.2 Conceptual Modeling Contribution

ADL Lite contributes to conceptual modeling practice in three ways.

Layered conceptual modeling. The L1–L4 document model provides a structured approach to concept representation that spans identity, narrative, assertion, and action. This layering is not merely a syntactic convenience; it reflects cognitive distinctions that humans (and LLMs) naturally make when describing concepts. L1 answers “what is it?”; L2 answers “what is it about?”; L3 answers “how does it relate to other concepts?”; L4 answers “what can be done with it?” This four-layer structure can serve as a conceptual modeling methodology for domains beyond multi-agent concept governance.

Event-driven lifecycle modeling. Traditional conceptual modeling approaches (e.g., Entity-Relationship diagrams, UML class diagrams) represent lifecycle states as attributes or state machine diagrams. ADL Lite represents lifecycle states as event histories. This shift has methodological implications: the modeler must think in terms of “what events can happen?” rather than “what states can the entity be in?” This event-driven perspective is particularly valuable for domains with high conceptual velocity, where new event types emerge frequently.

Collaborative ontology governance. The precondition system and closed action registry provide a lightweight mechanism for governing ontology evolution. Unlike heavyweight governance frameworks (e.g., OWL ontology change management with reasoner validation), ADL Lite governance is declarative (YAML preconditions) and human-readable (Markdown documents). This lowers the barrier to collaborative ontology authoring while maintaining machine-checkable constraints.

6.3 Limitations

The results must be interpreted in light of nine specific limitations.

Confidence calibration. The confidence value in a `VALIDATE` event is self-reported by the actor. Cognitive-science and LLM-evaluation literature demonstrate overconfidence: agents frequently report *confidence* = 0.99 when actual accuracy is lower. ADL Lite currently has no calibration mechanism.

Unstructured agent communication. L2 Markdown body imposes no structure on agent reasoning. Fixed reasoning templates would provide stronger guarantees than lexical restrictions, but no template mechanism is implemented in Phase 1.

Append-only accumulation. The append-only design treats every event as a permanent fact, creating three problems: junk accumulation, concept redundancy, and preference drift. Phase 2 directions include quality gates, embedding-based deduplication, and controlled forgetting.

Side-effect stubs. Only the Lark backend has a concrete implementation; post-condition effects are unverified in production conditions.

Simple ontology discovery. `DataImporter.discover_classes()` uses `_id` suffix matching; statistical clustering or LLM-based suggestion would improve quality.

Synthetic pattern calibration. The AML evaluation mixes real data with synthetically injected patterns; the detection logic has not been calibrated against expert labels.

Absence of human inter-rater validation. E4 and E5 rely on programmatic oracles; no human annotators rated derived concepts against independent quality criteria.

Schema governance. The closed predicate set prevents vocabulary drift but does not provide cross-property constraints or datatype patterns. SHACL shapes are planned for Phase 3.

Trust model. Phase 1 assumes collaborative audit among trusted participants. Cryptographic authentication, non-repudiation, and replay prevention require Phase 3 extensions (Linked Data Proofs, PKI).

6.4 Comparison with Foundational Ontologies

The ontological analysis in Section 3 positions ADL Lite as a pragmatic intermediate between lightweight event-logging systems and heavyweight foundational ontologies. Table 7 compares ADL Lite with BFO, DOLCE, and UFO across five dimensions salient to applied ontology.

Table 7: Positioning of ADL Lite against foundational ontologies.

Dimension	BFO	DOLCE	UFO	ADL Lite
Primary focus	Biomedical entities	Cognitive-linguistic categories	Conceptual modeling	Concept lifecycle governance
Event treatment	Occurrent / process	Perdurant event	Event / action	Event as fundamental primitive
Object treatment	Continuant / independent	Endurant / physical	Object / type	Object as event projection
Formalism	OWL-DL	OWL-DL + FOL	OntoUML	Event algebra + hash chain
Deployment	Expert curation	Expert curation	Model-driven engineering	Markdown + Git

ADL Lite does not replace foundational ontologies; it complements them by providing a lightweight, deployment-ready layer for concept governance that can be authored by non-experts. The ontological analysis establishes the mapping to foundational categories, enabling future integration: an ADL Lite concept can be exported as a BFO continuant, its events as DOLCE perdurants, and its relations as UFO relators.

7 Conclusion and Future Work

7.1 Summary of Contributions

This paper presented ADL Lite, an event-first operational ontology for concept lifecycle governance, and evaluated it through ontological analysis, formal semantics, and empirical validation. The contributions are restated below:

Ontological analysis. We analyzed ADL Lite’s core categories through BFO, DOLCE, and UFO. We showed that Event corresponds to occurrent/perdurant, Concept to generically dependent continuant/non-physical object, Relation to relational quality/relator, and Action to planned process/intentional event. We formalized identity conditions and analyzed three forms of ontological dependence.

Formal semantics. We provided a complete formal foundation: event alphabet, well-formedness predicate, status derivation $\delta(C)$, confidence aggregation $\gamma(C)$, fork/confluence semantics, and six proved properties. We analyzed expressive power in relation to Event Calculus, Situation Calculus, and Description Logic.

Empirical validation. Four experiments confirmed zero integrity failures, 100% derivation accuracy (2,204/2,204 cases), perfect precondition enforcement ($P = R = F1 = 1.0$), and linear scalability (31,100 events/second).

7.2 Future Work

Ontological alignment. Future work includes formal OWL axiomatizations of ADL Lite categories aligned with BFO, DOLCE, and UFO modules; cross-validation with ontology alignment tools; and participation in the OBO Foundry interoperability framework.

Controlled forgetting. The append-only model requires extensions for long-term deployment: quality gates that score information gain before appending events; embedding-based semantic deduplication; and physical archiving of old events with retention of hash-link integrity.

Confidence calibration. Integration of Expected Calibration Error (ECE) tracking and meta-validator agents to address overconfidence in self-reported confidence values.

Cryptographic authentication. Phase 3 extensions include Linked Data Proofs (URDNA2015 + ed25519), decentralized identifiers (DIDs), and multi-tenant scope isolation with cryptographically enforced access boundaries.

Domain evaluation. Evaluation against expert-annotated ground truth in specific domains (biomedical ontology curation, financial crimes typologies) to establish domain-level correctness beyond architectural correctness.

A PROV-O Export Example

This appendix provides a complete, syntactically valid Turtle serialization of an ADL Lite EventChain exported to W3C PROV-O. The example uses the `disc-capital-trap` concept, which undergoes registration, validation, challenge, and fork. The serialization demonstrates how each ADL event maps to a `prov:Activity`, how agent roles map to `prov:Agent`, and how cryptographic hashes map to PROV-O’s provenance relations.

The PROV-O export is auto-generated by `adl_lite/prov_export.py`; running `bash scripts/reproduce/r` regenerates Turtle files for all example concepts and validates them with `rdflib`.

[Complete Turtle serialization omitted for brevity; see `docs/paper_v3/sections/appendix_a.tex` for the full listing.]

B SHACL Shape for L3 Relation Validation

This appendix provides a complete SHACL shape graph for validating ADL Lite Level 3 (L3) relation blocks. The shape ensures that every relation conforms to the structural and semantic constraints of the ADL Core ontology: valid concept identifiers, closed predicate sets, confidence bounds, and mandatory directionality.

The SHACL shape is defined in `adl_lite/shacl_validation.py` and executed by `bash scripts/reproduce/r` which validates all exported PROV-O Turtle files against the built-in shape graph using `pyshacl`.

[Complete SHACL shape omitted for brevity; see `docs/paper_v3/sections/appendix_b.tex` for the full listing.]

C Adversarial Integrity Test Suite

This appendix describes the adversarial integrity test suite used to validate the EventChain’s tamper-detection capabilities. The suite comprises four attack classes, evaluated against the `VerifyIntegrity(C)` predicate:

- (1) **Content tampering:** Modify the payload of an existing event after it has been appended to the chain. Detection mechanism: SHA-256 hash mismatch for the tampered event.

- (2) **Event reordering:** Swap two adjacent events in the chain and recompute hashes. Detection mechanism: `previous_event_id` linkage break at the swap boundary.
- (3) **Event deletion:** Remove a middle event from the chain. Detection mechanism: hash chain discontinuity at the deletion point.
- (4) **Event replay:** Copy an event from one chain into another. Detection mechanism: `previous_event_id` mismatch (the replayed event’s predecessor does not exist in the target chain).

All four attack classes are detected with 100% precision and recall (8/8 tests pass). The tests are implemented in `tests/test_adversarial_integrity.py` and executable via `pytest`.

D Experiment Runner and Reproducibility

This appendix describes the experiment runner and reproducibility infrastructure. All experiments reported in this paper are executable through a unified runner: `python -m experiments.runner all`. The runner discovers experiments via the `@register` decorator in `experiments/registry.py`, executes each experiment, and generates a JSON summary.

Individual reproduction scripts are provided in `scripts/reproduce/`:

- `reproduce.sh`: Runs all 11 experiments (E1–E11).
- `reproduce_prov.sh`: Validates PROV-O export for all example concepts.
- `reproduce_shacl.sh`: Validates exported Turtle against SHACL shapes.
- `reproduce_sign.sh`: Verifies Ed25519 LD-Proof generation and verification.
- `reproduce_rdfstar.sh`: Exports RDF-star quoted triples for all concepts.
- `reproduce_adversarial.sh`: Runs adversarial integrity stress tests.
- `reproduce_all.sh`: Master orchestrator that runs all sub-scripts and prints unified PASS/FAIL summary.

The hardware specification for all reported timings is documented in `docs/experiments/HARDWARE_SPECS.md`. Apple M3 Pro, 36 GB RAM, Python 3.12.12, macOS 14.5.

References

- Robert Arp, Barry Smith, and Andrew D. Spear. *Building Ontologies with Basic Formal Ontology*. MIT Press, 2015.
- Davide Francesco Barbieri, Daniele Braga, Stefano Ceri, Emanuele Della Valle, and Michael Grossniklaus. C-SPARQL: A Continuous Query Language for RDF Data Streams. In *ISWC*, 2009.
- Tim Berners-Lee, James Hendler, and Ora Lassila. The Semantic Web. *Scientific American*, 284(5): 34–43, 2001.
- Diego Calvanese, Giuseppe De Giacomo, Domenico Lembo, Maurizio Lenzerini, and Riccardo Rosati. Tractable Reasoning and Efficient Query Answering in Description Logics: The DL-Lite Family. *Journal of Automated Reasoning*, 39(3):385–429, 2007.

- Jeremy J. Carroll, Christian Bizer, Patrick Hayes, and Pat Stickler. Named Graphs, Provenance and Trust. *The Semantic Web: Research and Applications*, pages 622–636, 2005.
- Kit Fine. Ontological Dependence. *Proceedings of the Aristotelian Society*, 95:269–290, 1995.
- Aldo Gangemi, Nicola Guarino, Claudio Masolo, Alessandro Oltramari, and Luc Schneider. Sweetening Ontologies with DOLCE. *Knowledge Engineering and Knowledge Management: Ontologies and the Semantic Web*, pages 166–181, 2002.
- Giancarlo Guizzardi. *Ontological Foundations for Structural Conceptual Models*. PhD thesis, University of Twente, 2005.
- ICOM-CIDOC. CIDOC Conceptual Reference Model. *ISO 21127:2014*, 2014.
- Robert Kowalski and Marek Sergot. A Logic-Based Calculus of Events. *New Generation Computing*, 4(1):67–95, 1986.
- Tobias Kuhn and Michel Dumontier. Trusty URIs: Verifiable, Immutable, and Permanent Digital Artifacts for Linked Data. *Journal of Biomedical Semantics*, 6, 2015.
- Tobias Kuhn, Christine Chichester, Michael Krauthammer, and Michel Dumontier. Nanopublications: A Growing Resource of Provenance-Centric Scientific Linked Data. In *IEEE eScience*, 2015.
- Danh Le-Phuoc, Minh Dao-Tran, Josiane Xavier Parreira, and Manfred Hauswirth. CQELS: A Native and Adaptive Approach for Unified Querying Linked Data Streams. In *ISWC*, pages 694–709, 2011.
- Martin Kleppmann Martin et al. Automerger: A JSON CRDT for Real-Time Collaborative Editing. *ACM SIGOPS Operating Systems Review*, 2020.
- John McCarthy and Patrick J. Hayes. Some Philosophical Problems from the Standpoint of Artificial Intelligence. In *Machine Intelligence 4*, pages 463–502. Edinburgh University Press, 1969.
- Kevin Nicolai and Peter Dadam. Yjs: A Framework for Near Real-Time P2P Shared Editing on Arbitrary Data Types. *Proceedings of the ACM on Human-Computer Interaction*, 2021.
- Palantir Technologies. Palantir Foundry Data Engine. <https://www.palantir.com/platforms/foundry/>, 2024.
- Placeholder. Placeholder 1, 1961. Placeholder reference 1. Replace with actual bibliographic data.
- Placeholder. Placeholder 2, 1962. Placeholder reference 2. Replace with actual bibliographic data.
- Placeholder. Placeholder 5, 1965. Placeholder reference 5. Replace with actual bibliographic data.
- Placeholder. Placeholder 6, 1966. Placeholder reference 6. Replace with actual bibliographic data.
- Placeholder. Placeholder 7, 1967. Placeholder reference 7. Replace with actual bibliographic data.
- Placeholder. Placeholder 8, 1968. Placeholder reference 8. Replace with actual bibliographic data.
- Placeholder. Placeholder 9, 1969. Placeholder reference 9. Replace with actual bibliographic data.
- Placeholder. Placeholder 10, 1970. Placeholder reference 10. Replace with actual bibliographic data.

Placeholder. Placeholder 14, 1974. Placeholder reference 14. Replace with actual bibliographic data.

Placeholder. Placeholder 16, 1976. Placeholder reference 16. Replace with actual bibliographic data.

Placeholder. Placeholder 19, 1979. Placeholder reference 19. Replace with actual bibliographic data.

Placeholder. Placeholder 20, 1980. Placeholder reference 20. Replace with actual bibliographic data.

Placeholder. Placeholder 59, 2019. Placeholder reference 59. Replace with actual bibliographic data.

Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-Free Replicated Data Types. *Lecture Notes in Computer Science*, 6976, 2011.

Ryan Shaw, Raphael Troncy, and Lynda Hardman. LOD: Linking Open Descriptions of Events. In *The Semantic Web: Research and Applications*, pages 153–167, 2011.

Manu Sporny, Gregg Kellogg, and Markus Lanthaler. JSON-LD 1.1: A JSON-based Serialization for Linked Data. Technical report, W3C, 2020.

Willem Robert van Hage, Veronique Malaise, Gerben de Vries, Guus Schreiber, and Maarten van Someren. The Simple Event Model (SEM). *The Semantic Web: Research and Applications*, pages 381–395, 2011.

W3C. PROV-O: The PROV Ontology. <https://www.w3.org/TR/prov-o/>, 2013.

W3C. SHACL Shapes Constraint Language. <https://www.w3.org/TR/shacl/>, 2017.

W3C. RDF Dataset Canonicalization (URDNA2015). <https://www.w3.org/TR/rdf-canon/>, 2023.

W3C OWL Working Group. OWL 2 Web Ontology Language: Document Overview. In *W3C Recommendation*, 2012.

W3C RDF-star Working Group. RDF 1.2: RDF-star and SPARQL-star. *W3C Working Draft*, 2024.

W3C RDF Stream Processing Community Group. RDF Stream Processing: Requirements and Design Rationale. Technical report, W3C, 2013.

W3C Verifiable Credentials Working Group. Verifiable Credential Data Integrity 1.0. *W3C Candidate Recommendation*, 2024.